

ADP470S

Applications Development Practice 4

TERM 2 COMPREHENSIVE STUDY NOTES

Topics Covered:

- Stack Operations & Recursion
- Queue Operations & Circular Queue
 - Priority Queue
- Binary Trees & Tree Traversals
- AVL / Height-Balanced Binary Trees
- Sorting Algorithms & Algorithm Analysis

1. Stack Operations & Recursion

1.1 What is a Stack?

A Stack is a linear data structure that follows the LIFO (Last In, First Out) principle. Think of a stack of plates — you can only add or remove from the top.

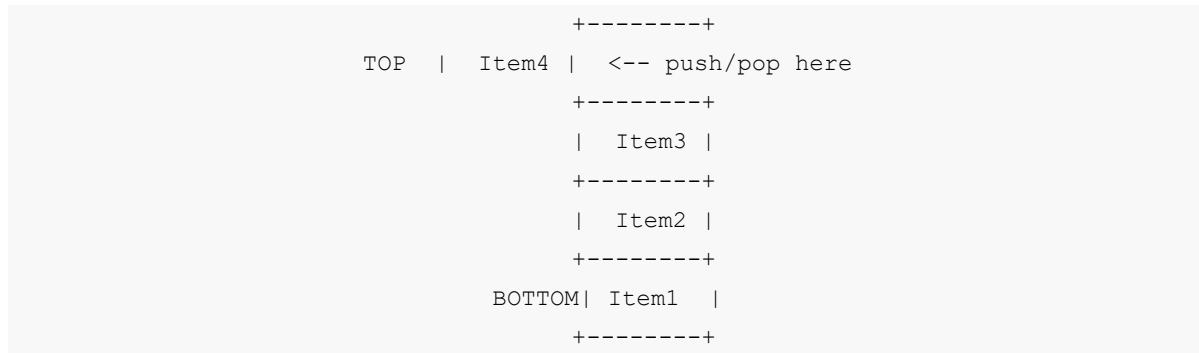


Figure 1.1 - Stack structure (LIFO)

Key Stack Properties

LIFO = Last In, First Out

push() = Add element to the top

pop() = Remove element from the top

peek() = View top element WITHOUT removing

isEmpty() = Check if stack is empty

isFull() = Check if stack is full (array-based)

1.2 Stack Operations in Java

```
public class Stack {  
    private int[] data;  
    private int top;  
    private int capacity;  
  
    public Stack(int size) {  
        data = new int[size];  
        capacity = size;  
        top = -1; // -1 means empty  
    }  
  
    // Push: add to top
```

```

public void push(int value) {
    if (isFull()) throw new RuntimeException("Stack Overflow!");
    data[++top] = value;
}

// Pop: remove from top
public int pop() {
    if (isEmpty()) throw new RuntimeException("Stack Underflow!");
    return data[top--];
}

// Peek: view top
public int peek() { return data[top]; }

public boolean isEmpty() { return top == -1; }
public boolean isFull() { return top == capacity - 1; }
}

```

1.3 How Recursion Uses the Call Stack

Every recursive method call is pushed onto the call stack. When the base case is reached, calls are popped off (unwound) in reverse order.

```

factorial(4) --> PUSHED
factorial(3) --> PUSHED
factorial(2) --> PUSHED
factorial(1) --> BASE CASE (returns 1)
factorial(2) <-- POPPED returns 2*1 = 2
factorial(3) <-- POPPED returns 3*2 = 6
factorial(4) <-- POPPED returns 4*6 = 24

```

Figure 1.2 - Call stack for factorial(4)

```

// Recursive factorial
public static int factorial(int n) {
    if (n == 0 || n == 1) return 1; // base case
    return n * factorial(n - 1);    // recursive call
}

// Iterative factorial (uses loop, no call stack growth)
public static int factorialIterative(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) result *= i;
}

```

```
    return result;
}
```

Property	Recursion	Iteration
Memory	Uses call stack (can overflow)	Fixed memory usage
Readability	Cleaner for tree/graph problems	Can be verbose
Performance	Overhead per call	Generally faster
Risk	StackOverflowError if too deep	No stack risk
Best for	Trees, graphs, divide & conquer	Simple loops

1.4 Stack Applications

- Expression evaluation (e.g., postfix/prefix calculation)
- Undo/redo operations in editors
- Browser back/forward navigation
- Balancing parentheses/brackets
- Depth-First Search (DFS) in graphs

2. Queue Operations & Circular Queue

2.1 What is a Queue?

A Queue is a linear data structure that follows FIFO (First In, First Out). Like a queue at a shop — the first person to join is served first.

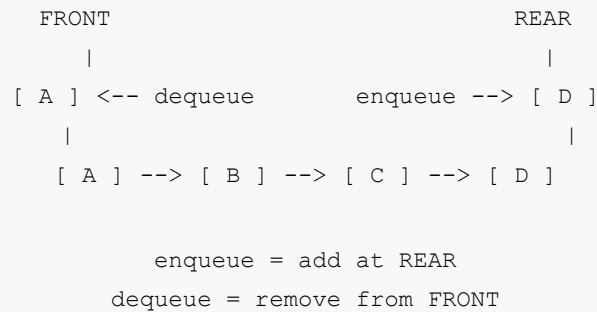


Figure 2.1 - Standard Queue (FIFO)

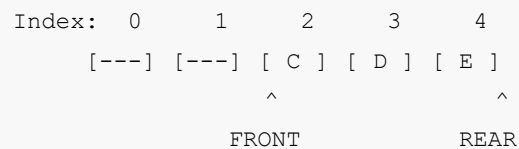
Queue Operations

- enqueue(x) = Add element x at the REAR
- dequeue() = Remove element from the FRONT
- peek() = View FRONT element without removing
- isEmpty() = Returns true if queue has no elements
- isFull() = Returns true if queue is at capacity

2.2 Problem with a Linear Queue

In a simple array-based queue, once elements are dequeued, the front pointer moves forward. This wastes memory because the empty slots at the front cannot be reused.

After several enqueue/dequeue operations:

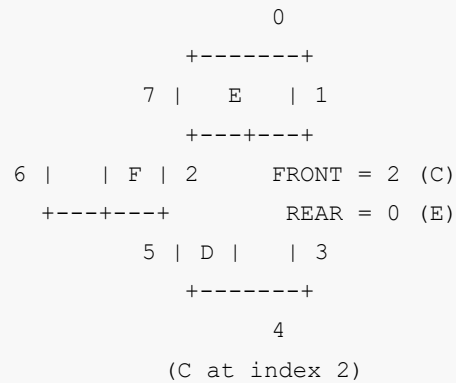


Slots 0 and 1 are WASTED -- cannot be reused!

Figure 2.2 - Wasted space in linear queue

2.3 Circular Queue

A Circular Queue solves the wasted space problem. The rear pointer wraps around to index 0 when it reaches the end of the array.



Rear wraps around: $(\text{rear} + 1) \% \text{capacity}$

Figure 2.3 - Circular Queue

```

public class CircularQueue {
    private int[] data;
    private int front, rear, size, capacity;

    public CircularQueue(int capacity) {
        this.capacity = capacity;
        data = new int[capacity];
        front = 0;
        rear = -1;
        size = 0;
    }

    public void enqueue(int value) {
        if (isFull()) throw new RuntimeException("Queue Full");
        rear = (rear + 1) % capacity; // wrap around!
        data[rear] = value;
        size++;
    }

    public int dequeue() {
        if (isEmpty()) throw new RuntimeException("Queue Empty");
        int val = data[front];
        front = (front + 1) % capacity; // wrap around!
        size--;
        return val;
    }
}
  
```

```
public boolean isEmpty() { return size == 0; }  
public boolean isFull() { return size == capacity; }  
}
```

2.4 Queue Linked List Implementation

An alternative is to implement a queue using a linked list — avoids the fixed-size problem entirely.

```
class Node {  
    int data;  
    Node next;  
    Node(int data) { this.data = data; }  
}  
  
public class QueueLL {  
    Node front, rear;  
  
    public void enqueue(int value) {  
        Node newNode = new Node(value);  
        if (rear != null) rear.next = newNode;  
        rear = newNode;  
        if (front == null) front = rear;  
    }  
  
    public int dequeue() {  
        if (front == null) throw new RuntimeException("Queue Empty");  
        int val = front.data;  
        front = front.next;  
        if (front == null) rear = null;  
        return val;  
    }  
}
```

3. Priority Queue

3.1 What is a Priority Queue?

A Priority Queue is a special queue where elements are served based on priority, not arrival order. Higher-priority elements are dequeued first.

Real-World Examples of Priority Queues

Hospital emergency room -- critical patients treated first

CPU task scheduling -- high-priority processes run first

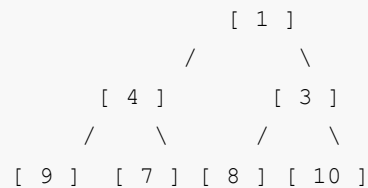
Dijkstra's algorithm -- always processes nearest node next

Print queue -- urgent documents printed first

3.2 Priority Queue using Min-Heap

The most efficient implementation of a priority queue uses a Binary Heap (usually a min-heap where the minimum element always has highest priority).

Min-Heap (smallest value = highest priority at root):



dequeue() always removes the ROOT (smallest = 1)

After removal, heap is restructured (heapify-down)

Figure 3.1 - Min-Heap Priority Queue

```
import java.util.PriorityQueue;

// Java's built-in Priority Queue is a MIN-heap by default
PriorityQueue<Integer> pq = new PriorityQueue<>();

pq.offer(5); // enqueue with priority
pq.offer(1);
pq.offer(3);

System.out.println(pq.poll()); // Output: 1 (lowest = highest priority)
System.out.println(pq.poll()); // Output: 3
```



```
System.out.println(pq.poll()); // Output: 5

// MAX-heap: reverse order
PriorityQueue<Integer> maxPQ = new PriorityQueue<>(Collections.reverseOrder());
```

3.3 Priority Queue Operations & Complexity

Operation	Description	Time Complexity
enqueue(x)	Insert element with priority	$O(\log n)$
dequeue()	Remove highest-priority element	$O(\log n)$
peek()	View highest-priority element	$O(1)$
isEmpty()	Check if empty	$O(1)$
size()	Return number of elements	$O(1)$

4. Binary Trees & Tree Traversals

4.1 Binary Tree Terminology

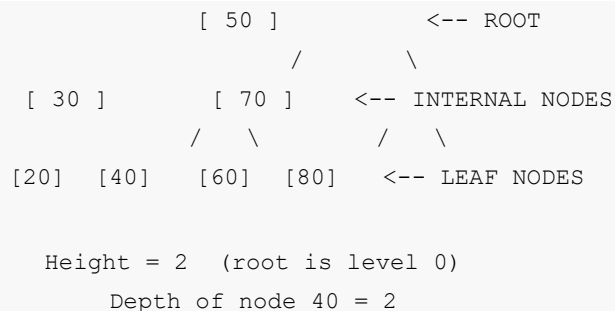


Figure 4.1 - Binary Tree Structure

Binary Tree Vocabulary

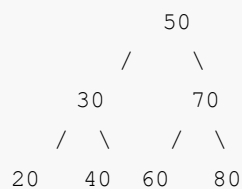
Root	= Top node (has no parent)
Leaf	= Node with no children
Parent	= Node directly above a node
Child	= Node directly below a parent
Height	= Longest path from root to leaf
Depth	= Distance from root to a node
Subtree	= A node and all its descendants
BST	= Binary Search Tree (left < root < right)

4.2 Binary Search Tree (BST) Property

In a BST, for every node:

- All values in LEFT subtree are LESS THAN the node
- All values in RIGHT subtree are GREATER THAN the node
- Both left and right subtrees are also BSTs

Insert: 50, 30, 70, 20, 40, 60, 80



Search 40: 40 < 50 -> go left -> 40 > 30 -> go right -> FOUND

Figure 4.2 - BST with insertions

```
class TreeNode {
    int value;
    TreeNode left, right;
    TreeNode(int value) { this.value = value; }
}

public class BST {
    TreeNode root;

    // Insert
    public TreeNode insert(TreeNode node, int val) {
        if (node == null) return new TreeNode(val);
        if (val < node.value) node.left = insert(node.left, val);
        else if (val > node.value) node.right = insert(node.right, val);
        return node;
    }

    // Search
    public boolean search(TreeNode node, int val) {
        if (node == null) return false;
        if (val == node.value) return true;
        if (val < node.value) return search(node.left, val);
        return search(node.right, val);
    }
}
```

4.3 Tree Traversals

Tree traversal means visiting every node exactly once. There are three standard depth-first traversals and one breadth-first traversal.

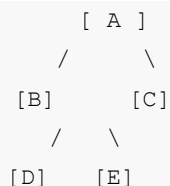


Figure 4.3 - Tree used in traversal examples below

4.3.1 In-Order Traversal (LEFT - ROOT - RIGHT)

Visits left subtree, then root, then right subtree. Produces SORTED output for a BST.

```

void inOrder(TreeNode node) {
    if (node == null) return;
    inOrder(node.left);        // 1. Left
    System.out.print(node.value + " "); // 2. Root
    inOrder(node.right);       // 3. Right
}

// Output for Figure 4.3: D -> B -> E -> A -> C

```

4.3.2 Pre-Order Traversal (ROOT - LEFT - RIGHT)

Visits root first, then left subtree, then right subtree. Useful for copying a tree.

```

void preOrder(TreeNode node) {
    if (node == null) return;
    System.out.print(node.value + " "); // 1. Root
    preOrder(node.left);               // 2. Left
    preOrder(node.right);              // 3. Right
}

// Output for Figure 4.3: A -> B -> D -> E -> C

```

4.3.3 Post-Order Traversal (LEFT - RIGHT - ROOT)

Visits left subtree, then right subtree, then root. Useful for deleting a tree.

```

void postOrder(TreeNode node) {
    if (node == null) return;
    postOrder(node.left);    // 1. Left
    postOrder(node.right);   // 2. Right
    System.out.print(node.value + " "); // 3. Root
}

// Output for Figure 4.3: D -> E -> B -> C -> A

```

4.3.4 Level-Order Traversal (Breadth-First Search)

Visits nodes level by level, left to right. Uses a Queue.

```

void levelOrder(TreeNode root) {
    if (root == null) return;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        TreeNode node = queue.poll();
        System.out.print(node.value + " ");
        if (node.left != null) queue.offer(node.left);
        if (node.right != null) queue.offer(node.right);
    }
}

```

```
}  
// Output for Figure 4.3: A -> B -> C -> D -> E
```

Traversal	Order	BST Output	Use Case
In-Order	Left, Root, Right	Sorted ascending	Sorted output
Pre-Order	Root, Left, Right	Root first	Copy / serialize tree
Post-Order	Left, Right, Root	Root last	Delete / evaluate tree
Level-Order	Level by level	BFS order	Shortest path in tree

5. AVL / Height-Balanced Binary Tree

5.1 The Problem with Unbalanced BSTs

A regular BST can degrade into a linked list if elements are inserted in sorted order. This makes all operations $O(n)$ instead of $O(\log n)$.

Inserting 10, 20, 30, 40, 50 into a regular BST:

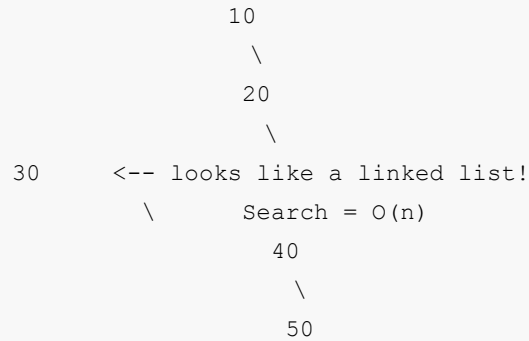


Figure 5.1 - Degenerate (unbalanced) BST

5.2 What is an AVL Tree?

An AVL Tree (Adelson-Velsky and Landis, 1962) is a self-balancing BST where the heights of the two child subtrees of any node differ by at most 1 (Balance Factor).

AVL Tree Key Concept

Balance Factor (BF) = $\text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$

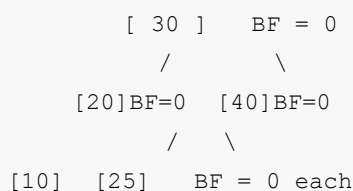
BF must be -1, 0, or +1 for EVERY node.

If BF becomes -2 or +2, the tree is UNBALANCED -> rotations needed.

Height of empty subtree = -1

Height of leaf node = 0

Balanced AVL Tree:



All Balance Factors are -1, 0, or +1 --> VALID AVL Tree

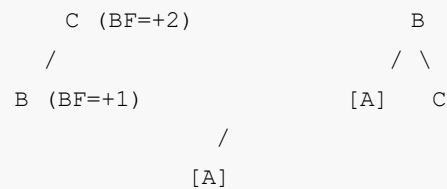
Figure 5.2 - Valid AVL Tree

5.3 AVL Rotations

When an insertion or deletion causes a node's BF to become +2 or -2, we restore balance using rotations.

5.3.1 Right Rotation (LL Case - Left-Left imbalance)

Before (unbalanced): After Right Rotation:



Trigger: Left child's Left subtree is too tall

Figure 5.3 - Right Rotation

5.3.2 Left Rotation (RR Case - Right-Right imbalance)

Before (unbalanced): After Left Rotation:

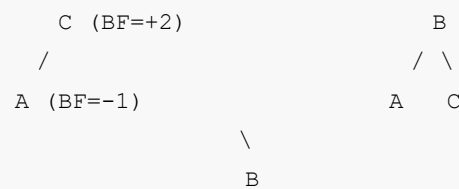


Trigger: Right child's Right subtree is too tall

Figure 5.4 - Left Rotation

5.3.3 Left-Right Rotation (LR Case)

Before: After LR Rotation:



Step 1: Left rotate on A

Step 2: Right rotate on C

Figure 5.5 - Left-Right Rotation

5.3.4 Right-Left Rotation (RL Case)

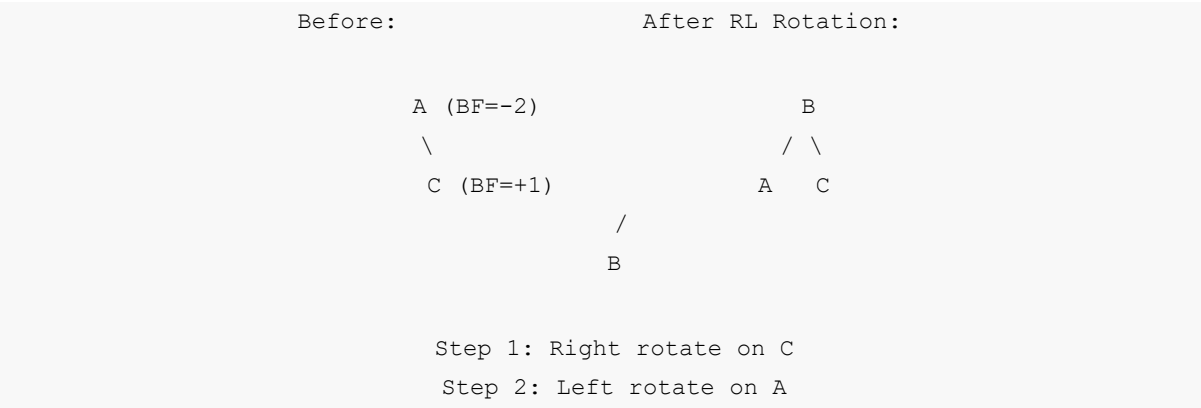


Figure 5.6 - Right-Left Rotation

Case	Imbalance	Fix
LL	Left child, Left subtree tall	Single Right Rotation
RR	Right child, Right subtree tall	Single Left Rotation
LR	Left child, Right subtree tall	Left then Right Rotation
RL	Right child, Left subtree tall	Right then Left Rotation

5.4 AVL vs BST Performance

Operation	BST (Worst Case)	AVL Tree (All Cases)
Search	O(n) — degenerate	O(log n) — always
Insert	O(n)	O(log n)
Delete	O(n)	O(log n)
Space	O(n)	O(n)

6. Sorting Algorithms & Algorithm Analysis

6.1 Overview of Sorting Algorithms in Scope

Algorithm	Best Case	Average Case	Worst Case	Space	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

6.2 Heap Sort

Heap Sort uses a Binary Max-Heap data structure. Elements are first arranged into a max-heap, then repeatedly extracted to produce a sorted array.

Heap Sort Steps

Phase 1 - Build Max-Heap: Convert the array into a max-heap (largest at root)

Phase 2 - Extract: Swap root (max) with last element, reduce heap size, heapify

Repeat Phase 2 until heap size = 1 --> array is sorted

```
Array: [4, 10, 3, 5, 1]

Step 1: Build Max-Heap:
      10
     /  \
    5    3
   /  \
  4    1

Step 2: Swap 10 with last (1), heapify:
Heap: [5, 4, 3, 1] | Sorted: [10]

Step 3: Swap 5 with last (1), heapify:
Heap: [4, 1, 3] | Sorted: [5, 10]

Result: [1, 3, 4, 5, 10]
```

Figure 6.1 - Heap Sort walkthrough

```

public static void heapSort(int[] arr) {
    int n = arr.length;

    // Build max-heap
    for (int i = n/2 - 1; i >= 0; i--) heapify(arr, n, i);

    // Extract elements from heap
    for (int i = n-1; i > 0; i--) {
        int temp = arr[0]; arr[0] = arr[i]; arr[i] = temp; // swap
        heapify(arr, i, 0);
    }
}

static void heapify(int[] arr, int n, int i) {
    int largest = i, left = 2*i+1, right = 2*i+2;
    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;
    if (largest != i) {
        int temp = arr[i]; arr[i] = arr[largest]; arr[largest] = temp;
        heapify(arr, n, largest);
    }
}

```

6.3 Insertion Sort

Builds the sorted array one element at a time by picking each element and inserting it in its correct position. Efficient for small or nearly-sorted data.

Array: [5, 3, 8, 4, 2]

Pass 1: [3, 5, 8, 4, 2] (3 inserted before 5)
 Pass 2: [3, 5, 8, 4, 2] (8 already in position)
 Pass 3: [3, 4, 5, 8, 2] (4 inserted between 3 and 5)
 Pass 4: [2, 3, 4, 5, 8] (2 inserted at beginning)

Figure 6.2 - Insertion Sort walkthrough

```

public static void insertionSort(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i];
        int j = i - 1;

        // Shift elements greater than key to the right
        while (j >= 0 && arr[j] > key) {

```

```

        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] = key;
}
}

```

6.4 Merge Sort

A divide-and-conquer algorithm. Recursively splits the array in half, sorts each half, then merges them back together. Always $O(n \log n)$.

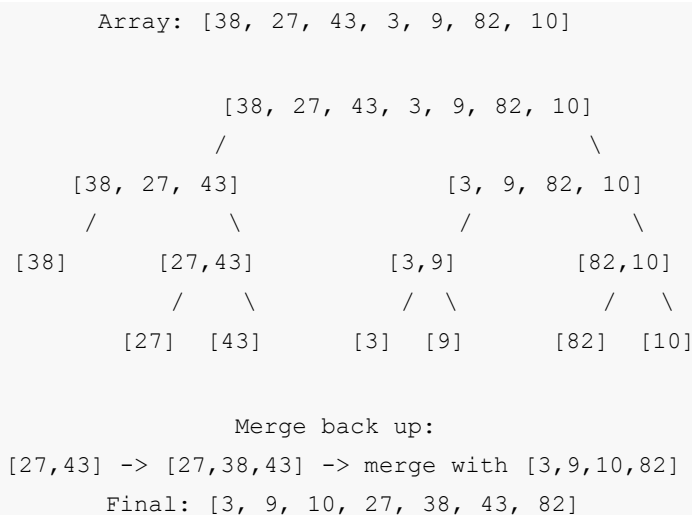


Figure 6.3 - Merge Sort divide and conquer

```

public static void mergeSort(int[] arr, int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);      // sort left half
        mergeSort(arr, mid + 1, right); // sort right half
        merge(arr, left, mid, right);   // merge halves
    }
}

static void merge(int[] arr, int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int[] L = new int[n1], R = new int[n2];
    for (int i=0; i<n1; i++) L[i]=arr[l+i];
    for (int j=0; j<n2; j++) R[j]=arr[m+1+j];
    int i=0, j=0, k=l;
    while (i<n1 && j<n2) arr[k++] = (L[i]<=R[j]) ? L[i++] : R[j++];
}

```

```

while (i<n1) arr[k++]=L[i++];
while (j<n2) arr[k++]=R[j++];
}

```

6.5 Quick Sort

Picks a pivot element and partitions the array around it. Elements less than pivot go left, greater go right. Recursively sort each side. Average $O(n \log n)$ but $O(n^2)$ worst case.

Array: [10, 7, 8, 9, 1, 5] Pivot = 5 (last element)

Partition step:

Compare each element to pivot (5):

1 < 5 -> left side: [1, 5, ...]

After partition: [1, 5, 10, 7, 8, 9] pivot 5 is in correct position

Recursively sort [1] and [10, 7, 8, 9]

Final: [1, 5, 7, 8, 9, 10]

Figure 6.4 - Quick Sort partition example

```

public static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pivotIdx = partition(arr, low, high);
        quickSort(arr, low, pivotIdx - 1); // sort left
        quickSort(arr, pivotIdx + 1, high); // sort right
    }
}

static int partition(int[] arr, int low, int high) {
    int pivot = arr[high], i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
        }
    }
    int tmp = arr[i+1]; arr[i+1] = arr[high]; arr[high] = tmp;
    return i + 1;
}

```

6.6 Selection Sort

Finds the minimum element repeatedly and places it at the beginning. Simple but always $O(n^2)$ regardless of input.

Array: [64, 25, 12, 22, 11]

Pass 1: Find min (11), swap with arr[0]: [11, 25, 12, 22, 64]
Pass 2: Find min in [25,12,22,64] -> 12, swap: [11, 12, 25, 22, 64]
Pass 3: Find min in [25,22,64] -> 22, swap: [11, 12, 22, 25, 64]
Pass 4: Find min in [25,64] -> 25, swap: [11, 12, 22, 25, 64]
Result: [11, 12, 22, 25, 64]

Figure 6.5 - Selection Sort walkthrough

```
public static void selectionSort(int[] arr) {
    for (int i = 0; i < arr.length - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < arr.length; j++) {
            if (arr[j] < arr[minIdx]) minIdx = j;
        }
        int tmp = arr[minIdx]; arr[minIdx] = arr[i]; arr[i] = tmp;
    }
}
```

6.7 Bubble Sort (Review + Matrix Multiplication)

Bubble Sort repeatedly compares adjacent elements and swaps them if in wrong order. The largest element 'bubbles up' to the end each pass.

```
public static void bubbleSort(int[] arr) {
    for (int i = 0; i < arr.length - 1; i++) {
        boolean swapped = false;
        for (int j = 0; j < arr.length - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int tmp = arr[j]; arr[j] = arr[j+1]; arr[j+1] = tmp;
                swapped = true;
            }
        }
        if (!swapped) break; // already sorted (Best case  $O(n)$ )
    }
}
```

Matrix Multiplication using Bubble Sort context

Matrix multiplication uses nested loops — a key example of $O(n^3)$ complexity:

```
// Multiply matrix A (n x m) by B (m x p) -> C (n x p)
```

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < p; j++) {
        C[i][j] = 0;
        for (int k = 0; k < m; k++) {
            C[i][j] += A[i][k] * B[k][j]; // O(n^3) overall
        }
    }
}

```

6.8 Algorithm Analysis Summary

Best / Worst / Average Case Definitions

Best Case = Minimum possible operations (e.g., array already sorted)

Worst Case = Maximum operations (e.g., array in reverse order)

Average Case = Expected operations over all possible inputs

Big O Notation: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$

Algorithm	When to Use
Insertion Sort	Small or nearly-sorted arrays (very fast in practice)
Merge Sort	Large arrays, need guaranteed $O(n \log n)$, linked lists
Quick Sort	General purpose, average $O(n \log n)$, in-place sorting
Heap Sort	Need worst-case $O(n \log n)$ with $O(1)$ extra space
Bubble Sort	Learning / detecting sorted arrays (not practical)
Selection Sort	Minimise number of swaps

7. Quick Reference & Exam Tips

7.1 Traversal Memory Aid

Mnemonic for Tree Traversals

In-Order = L - N - R (Left, Node, Right) --> 'In a BST = sorted'

Pre-Order = N - L - R (Node, Left, Right) --> 'Root PRE(first)'

Post-Order = L - R - N (Left, Right, Node) --> 'Root is POST(last)'

7.2 AVL Rotation Decision Table

BF of unbalanced node	BF of its child	Rotation needed
+2 (left heavy)	+1 or 0 (left heavy)	Single Right Rotation (LL)
+2 (left heavy)	-1 (right heavy)	Left-Right Rotation (LR)
-2 (right heavy)	-1 or 0 (right heavy)	Single Left Rotation (RR)
-2 (right heavy)	+1 (left heavy)	Right-Left Rotation (RL)

7.3 Stack vs Queue vs Priority Queue

Structure	Order	Key Operations	Example Use
Stack	LIFO	push, pop, peek	Browser back button, Recursion
Queue	FIFO	enqueue, dequeue	Print queue, BFS
Circular Queue	FIFO + wrap-around	$(\text{rear}+1)\% \text{capacity}$	CPU scheduling buffer
Priority Queue	By priority	offer, poll	Dijkstra's, Hospital triage

7.4 Important Formulas

- Circular Queue: next index = $(\text{current} + 1) \% \text{capacity}$
- Balance Factor = $\text{height}(\text{left}) - \text{height}(\text{right})$
- Height of complete binary tree with n nodes = $\text{floor}(\log_2(n))$
- In a max-heap, parent of node i = $\text{floor}((i-1)/2)$
- Left child of node i = $2i+1$, Right child = $2i+2$

Term 2 Exam Checklist

- ☐ Can explain LIFO vs FIFO with examples
- ☐ Can trace Stack push/pop operations

- ☐ Can implement Circular Queue with wrap-around formula
- ☐ Can perform In-Order, Pre-Order, Post-Order traversals by hand
- ☐ Can calculate Balance Factor of each node in an AVL tree
- ☐ Can identify which AVL rotation to apply (LL, RR, LR, RL)
- ☐ Can trace Heap Sort, Insertion Sort, Merge Sort, Quick Sort
- ☐ Know Big O complexity for best/average/worst for each sort
- ☐ Understand the difference between stable and unstable sorts